Adaptive NPC Behavior with Reinforcement Learning in 2D Game

Binh Vu Hoa[1], Long Pham Hoang[1]
Email: hoabinh105.work@gmail.com, phamhoanglong041003@gmail.com

ABSTRACT*—In this paper, we apply the Proximal Policy Optimization (PPO) algorithm to train Non-Player Characters (NPCs) with adaptive behavior in a 2D role-playing game (2D RPG). The model is formulated as a Markov Decision Process (MDP), optimizing the action policy through PPO and adjusting the reward function (reward shaping) to enhance learning capability and stability in environments with sparse feedback, in which NPC actions and states reflect survival factors such as food, water, sleep, and stress. To increase the naturalness of NPC behavior, we integrate Behavior Cloning (BC), which allows the model to learn from human behavioral data before continuing training with PPO. The experimental results show that incorporating imitation learning increases survival time by approximately 29.46% compared to the non-integrated model, while maintaining training stability thanks to PPO's clipping mechanism and BC-based fine-tuning. These results provide a foundation for developing adaptive game AI and expanding into multi-agent environments.*

Keywords—*Reinforcement Learning, Proximal Policy Optimization (PPO), NPC, Reward Shaping, Markov (MDP), Behavior Cloning (BC).*

I. INTRODUCTION

In the context of the modern gaming industry, which increasingly emphasizes interactivity and artificial intelligence, developing NPCs capable of self-learning and adaptive behavior has become an important research direction. Reinforcement Learning (RL), especially deep learning–based models, has been widely adopted to create NPCs with flexible and natural behaviors compared to traditional rule-based control systems [1].

Deep Q-Network (DQN), proposed by Mnih et al. (2015), combines Q-Learning with deep neural networks, in which the network is used to approximate the action-value function (Q-function) [2]. This model enables agents to learn policies directly from raw image inputs in Atari games, thereby extending the applicability of Reinforcement Learning to environments with large state spaces, unstructured inputs, and nonlinear relationships between states, actions, and rewards [3].

Subsequent studies expanded DQN for shooter-game NPCs, showing that behavioral performance increased by approximately 40% compared to rule-based systems in combat scenarios [4]. Alongside this, Policy Gradient approaches and Transformer-based networks were applied to Roguelike games, enabling NPCs to learn player styles and react more flexibly to player actions [5]. Furthermore, multi-agent RL research has focused on training companion NPCs, allowing them to coordinate with players and adapt to group objectives [6].

Notably, recent work on Dynamic NPC AI and sparse-reward RL has leveraged the Proximal Policy Optimization (PPO) algorithm to improve stability and convergence in NPC behavior training [7] [8] [9]. The study "Leveraging Deep Reinforcement Learning for Dynamic NPC Behavior in Unity3D" [10] demonstrated that PPO can enhance training efficiency and improve player experience in Unity environments.

However, most of these studies primarily focus on 3D environments or single-task combat behavior, while 2D RPG settings—where NPCs must balance combat, survival (food, sleep, stress, etc.), and limited resource management—remain relatively underexplored.

In this paper, we focus on developing adaptive NPC models in 2D RPG games based on PPO combined with Behavior Cloning (BC), with the following objectives: (i) modeling the problem as a Markov Decision Process (MDP) and implementing methods from Q-Learning, DQN, to PPO; (ii) optimizing action policies through improved state and reward design to enhance convergence speed and training stability for survival-related tasks; and (iii) evaluating effectiveness through NPC behavior simulations, parameter comparisons, and learning-result analysis.

The structure of the paper is organized as follows. Section 1 presents the theoretical background and PPO algorithms. Section 2 provides the theoretical foundations of BC and PPO. Section 3 describes the experimental results and analysis of NPC behavior with BC fine-tuning. Section 4 presents the final experimental results, followed by the conclusion.

II. THEORETICAL FOUNDATIONS OF PROXIMAL POLICY OPTIMIZATION

Next, we present the theoretical background related to the Proximal Policy Optimization (PPO) algorithm in deep reinforcement learning. Reinforcement Learning (RL) is built upon the Markov Decision Process (MDP), which serves as the fundamental mathematical framework for describing the interaction between an agent and its environment [1]. Based on this foundation, the PPO algorithm is introduced as an efficient policy optimization method designed to improve training stability and performance compared to traditional approaches within the Policy Gradient family [2].

A. Markov Decision Process (MDP)

The game environment is modeled as a Markov Decision Process (MDP), consisting of five components [11]:

- S: the state space
- A: the set of possible actions (movement, attack, resting)
- $P(s'|s,a)$: the state transition probability
- $R(s,a)$: the reward function
- γ: the discount factor, representing the priority of future rewards

The objective of the agent is to find an optimal policy $\pi^*(a|s)$ that maximizes the long-term expected return:

$$J_\pi = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right] \tag{1}$$

where R_t denotes the reward at time step t [1] [12].

The MDP framework serves as the basis for RL algorithms such as Q-Learning, Actor–Critic, and PPO, which differ primarily in how they estimate and update policies [1], [11]. In the context of 2D RPG environments, MDP enables the modeling of dynamic survival factors and supports PPO in learning adaptive behavior.

B. Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) là một thuật toán trong nhóm Policy Gradient hiện đại, được đề xuất bởi Schulman et al. (2017) [2], nhằm giải quyết vấn đề bất ổn khi tối ưu chính sách trực tiếp trong học tăng cường sâu. PPO kế thừa tính ổn định của TRPO (Trust Region Policy Optimization) [6] nhưng đơn giản hơn, dễ huấn luyện hơn và phù hợp với các môi trường có phản hồi khan hiếm (sparse rewards) — như các trò chơi nhập vai (RPG) [1].

C. Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) is a modern algorithm in the Policy Gradient family, proposed by Schulman et al. (2017) [2], designed to address instability issues when directly optimizing policies in deep reinforcement learning. PPO inherits the stability of TRPO (Trust Region Policy Optimization) [6] while being simpler, easier to train, and suitable for environments with sparse rewards—such as role-playing games (RPGs) [1].

1) Theoretical Foundations

In Policy Gradient methods, the objective is to maximize the long-term expected return [1]:

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right] \tag{2}$$

The gradient of this objective with respect to the policy parameters θ is estimated using the REINFORCE formula [9]:

$$\nabla_\theta J(\theta) = \mathbb{E}_t \left[\nabla_\theta \log \pi_\theta(a_t|s_t)\, A_t \right] \tag{3}$$

where A_t is the advantage function, which measures how much better or worse an action is compared to the expected behavior at time step t.

To reduce variance, the advantage is often estimated using Generalized Advantage Estimation (GAE) [10]:

$$A_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l\, \delta_{t+l}, \quad \delta_t = R_t + \gamma V(s_{t+1}) - V(s_t), \quad \lambda \in [0,1] \tag{4}$$

where λ is a parameter controlling the bias–variance trade-off.

2) PPO Objective Function

Unlike TRPO, PPO does not impose a complex KL-divergence constraint but instead uses a *clipped surrogate objective* to limit the policy update ratio [2]:

$$\mathcal{L}_{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \big(r_t(\theta) A_t,\ \text{clip}\big(r_t(\theta),\, 1-\epsilon,\, 1+\epsilon\big) A_t \big) \right] \tag{5}$$

where:

- $r_t(\theta) = \dfrac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}$ is the probability ratio,
- ϵ (typically from 0.2 to 0.8) stabilizes training and prevents overfitting/underfitting,
- The clip() function prevents the gradient from exceeding the trust region, avoiding abrupt policy shifts.

PPO performs multiple epochs of updates on the same trajectory, improving sample efficiency [2]. The PPO-Penalty variant additionally incorporates a KL-divergence term:

$$\mathcal{L}_{\text{KL-PEN}}(\theta) = \mathbb{E}_t \left[r_t(\theta) A_t - \beta\, D_{\text{KL}} \left(\pi_{\theta_{\text{old}}} \parallel \pi_\theta \right) \right] \tag{6}$$

where the coefficient β is automatically adjusted to maintain a stable policy distance [2], [13].

3) Actor–Critic Architecture in PPO

PPO operates within the Actor–Critic framework, consisting of two neural networks [1]:

- Actor (π_θ): outputs the probability distribution over actions.
- Critic (V_ϕ): estimates the state value for computing the advantage.

The combined loss function is:

$$L(\theta, \phi) = L_{\text{clip}}(\theta) + c_1\, L_V(\phi) - c_2\, S[\pi_\theta] \tag{7}$$

where:

- L_V is the squared error between the predicted and target value (value loss),
- $S[\pi_\theta]$ is the entropy regularization term that preserves action diversity,
- c_1, c_2 are weighting coefficients.

D. Behavior Cloning (BC) and Integration with PPO

1) Foundations of Behavioral Cloning (BC)

Behavioral Cloning (BC) is a fundamental method in imitation learning, where the agent learns a policy by mimicking expert behavior through supervised learning. Unlike traditional Reinforcement Learning (RL) — in which the agent explores the environment and receives feedback via reward signals — BC directly maps observed states to expert actions using a demonstration dataset:

$$D = \{(s_i, a_i)\}_{i=1}^{N},$$

collected from an expert policy π_E [12].

The learning objective of BC is to find a policy π_θ with parameters θ that minimizes the supervised loss function on the dataset:

$$L(\theta) = \frac{1}{N} \sum_{i=1}^{N} \ell\big(\pi_\theta(s_i), a_i\big), \tag{8}$$

where ℓ is typically the cross-entropy loss for discrete action spaces or mean squared error for continuous control. Despite its simplicity and ease of implementation — particularly in high-dimensional state spaces such as image-based control — BC remains a strong and useful baseline when safe exploration is required or when abundant expert data is available [12].

2) Limitations and Solutions

Behavioral Cloning (BC) suffers from covariate shift: states outside the demonstrations lead to compounding errors (e.g., sudden spikes in stress in RPG environments). A solution is DAgger, which augments the dataset [14]. In this work, BC is used as a pre-training step to initialize a human-like policy, thereby reducing risks when combined with PPO.

3) Combining BC with PPO for Fine-Tuning

To improve natural behavior under sparse rewards, BC is used to pre-train before PPO: (i) training on 412,641 steps collected from the Unity recorder; (ii) fine-tuning PPO with a warm start, incorporating entropy regularization [12].

Specifically, the overall hybrid loss is defined as:

$$L_{\text{total}} = L_{\text{PPO}} + \alpha L_{\text{BC}} + \beta S[\pi_\theta], \tag{9}$$

where the balancing coefficients are $\alpha = 0.5$ to 1 and $\beta = 0.05$ to 0.1. In the early stages, we want the agent to closely follow the collected demonstrations, so we set $\alpha = 1$ and $\beta = 0.05$ to ensure that the agent prioritizes BC when making decisions. After every 100,000 steps, we gradually decrease α to 0.5 and increase β to 0.1, allowing the agent to explore strategies beyond the dataset. This is necessary because, in a complex environment, the agent is likely to get stuck if not sufficiently encouraged to explore.

III. PROBLEM DESIGN AND APPLICATION MODEL

A. *Problem Modeling*

1) Game Description

The Life-Simulation game we designed is a grid-based environment that simulates daily human survival behaviors, including eating, drinking, sleeping, working, and entertainment. The map is structured with household objects arranged as obstacles between interaction points, requiring the agent to navigate around them when performing actions. The objective of the machine-controlled character (NPC) is to overcome these obstacles, reach the required interaction points, and simultaneously optimize its survival indicators.

The NPC's survival indicators include:

- **Food**: Hunger level
- **Drink**: Thirst level
- **Stress**: Stress level
- **Sleep**: Sleepiness level
- **Money**: Amount of money

At each movement step, the NPC loses a certain amount of its survival indicators. Additionally, each interaction requires specific conditions to realistically simulate daily life. Specifically:

- To perform eating or drinking actions, the NPC must have a required amount of money.
- To perform working actions, the NPC must have sufficient physical condition (hunger, thirst, and sleep levels), and each work session slightly increases stress.
- Whenever the NPC's hunger, thirst, or sleep indicators fall below a certain threshold, its stress level gradually increases over time.

The character loses the game if the survival indicators reach critical thresholds: ≥ 72 for *Stress*, or ≤ 0 for any of the remaining indicators.

Hình 1. Main interface of the game

2) State Space

The state space S is represented as a real-valued vector consisting of both the NPC's survival indicators and spatial information of the environment. Specifically, each state includes the NPC's current coordinates (x, y) within the environment; survival indicators such as sleep, hunger, thirst, stress, and the amount of money currently possessed; as well as the time of day, discretized into 1440 steps per day (with each hour corresponding to 60 steps). This discretization enables the agent to learn time-dependent behaviors (e.g., sleeping during nighttime and working during daytime).

Additionally, to provide richer contextual information about the environment, we incorporate distances between the agent and key objects such as the stove, refrigerator, door, bed, and relaxation area.

3) Action Space

The environment is defined with 15 discrete actions, divided into two branches. The movement branch consists of 9 directional actions (up, down, left, right, up-left, up-right, down-left, down-right) and an idle action. The interaction branch consists of 6 actions: eating, drinking, sleeping, resting, entertaining, and doing nothing.

4) Episode Termination

An episode terminates when the agent reaches an unsustainable survival condition—specifically, when the food or drink indicator satisfies $x \leq 0$, stress satisfies $x \geq 72$, or sleep satisfies $x \leq 0$. When this condition occurs, the agent receives a penalty of -1.0 reward as a form of negative reinforcement, encouraging the agent to avoid entering a "game over" state.

5) Reward Function

The reward function is designed to encourage balanced survival behavior—providing positive rewards when the agent maintains healthy survival indicators and penalizing inefficient or passive actions.

Condition	Description	Reward
R1	Gradual penalty based on survival indicators	$-0.01 \times$ (hunger + thirst + tired + stress)
R2	Strong penalty when any indicator reaches a danger threshold	-0.05 per indicator exceeding the threshold
R3	Agent dies	-5
R4	Reaching an interaction point and successfully performing an action (eat, drink, sleep, work, relax)	$+0.04$, $+0.02$, $+0.1$, $+0.04$, $+0.03$
R5	Performing an interaction when indicators are appropriate	$+0.05$
R6	Performing an interaction when indicators are inappropriate	-0.02
R7	Maintaining all indicators at healthy levels	$+0.4$

All indicators (*hunger*, *thirst*, *tired*, *stress*) are normalized to the range $[0, 1]$ via min–max scaling. These reward terms can be grouped into two categories:

Process-based rewards: R1, R2, R7 — discouraging the agent from letting survival indicators deteriorate, while encouraging stable maintenance.

Outcome-based rewards: R3–R6 — guiding the agent toward productive interactions and effective task execution.

The following table shows the model's performance when removing one reward condition at a time, using comparable steps and state distributions:

R1	R2	R3	R4	R5	R6	R7	Max step	Min step	Mean step
No	Yes	Yes	Yes	Yes	Yes	Yes	3096	224	1073.175
Yes	No	Yes	Yes	Yes	Yes	Yes	3002	127	1030.12
Yes	Yes	No	Yes	Yes	Yes	Yes	1836	168	721.6389
Yes	Yes	Yes	No	Yes	Yes	Yes	1798	200	725.75
Yes	Yes	Yes	Yes	No	Yes	Yes	3101	214	1080.1751
Yes	Yes	Yes	Yes	Yes	No	Yes	2803	129	910.613
Yes	Yes	Yes	Yes	Yes	Yes	No	1721	117	680.1751
Yes	Yes	Yes	Yes	Yes	Yes	Yes	4102	122	1316.1341

As shown, within the process-based group, condition R7 has the strongest impact on model performance; removing it significantly degrades survival time. Conditions R1 and R2 are less influential but still beneficial. Within the outcome-based group, every condition contributes noticeably to the agent's overall learning quality.

B. PPO + BC Model

1) Input Representation

At each time step t, the agent observes a continuous state vector:

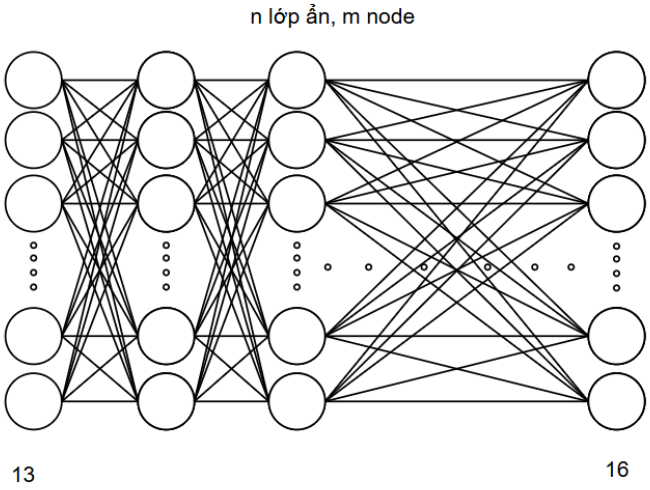
$$s_t = [x_1, x_2, \ldots, x_{n_{\text{features}}}]^\top \in \mathbb{R}^{n_{\text{features}}}, \tag{10}$$

where n_{features} denotes the total number of observed features (e.g., position, HP, energy, food, drink, stress, etc.). The input shape is therefore (n_{features}).

2) Network Architecture

Both the Actor and Critic share a feedforward fully-connected neural network (with an optional shared encoder followed by separate branches for Actor and Critic). The configuration used in our experiments is summarized as follows:

- **Input layer**: 13 neurons ($n_{\text{features}} = 13$).
- **Hidden layers**: n layers, each with 128 neurons, activation = ReLU, where $\text{ReLU}(x) = \max(0, x)$.
- **Output layers**:
  - Actor: 9 movement actions + 6 interaction actions.
  - Critic: a single scalar neuron estimating the value function $V_\phi(s)$.



Hình 2. Neural network architecture (Actor, Interaction Head, and Critic).

3) Actor Output (Action Distribution)

Let $z(s) = [z_1, \ldots, z_{|\mathcal{A}|}]$ denote the logits produced by the Actor network. The action probability distribution is computed via the softmax function:

$$\pi_\theta(a_i \mid s) = \frac{\exp(z_i(s))}{\sum_{j=1}^{|\mathcal{A}|} \exp(z_j(s))}, \quad i = 1, \ldots, |\mathcal{A}|. \tag{11}$$

The agent samples actions according to:

$$a_t \sim \pi_\theta(\cdot \mid s_t).$$

In practice, exploration may be adjusted using ε-greedy sampling or temperature-scaled softmax.

4) Critic Output (State Value)

The Critic returns a scalar:

$$V_\phi(s_t) \approx \mathbb{E}[G_t \mid s_t], \qquad G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}, \tag{12}$$

where G_t is the return starting from time step t.

5) Loss Functions and Update

PPO-Clip is used for the Actor, MSE for the Critic, and an entropy bonus for exploration.

i) *Actor Objective (clipped surrogate)*

$$L_{\text{actor}}(\theta) = \mathbb{E}_t \left[\min\big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\big) \right], \tag{13}$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ and $\hat{A}_t$ is the estimated advantage (using GAE).

ii) *Critic Loss (value loss)*

$$L_{\text{value}}(\phi) = \mathbb{E}_t\big[(V_\phi(s_t) - R_t^{\text{target}})^2\big], \tag{14}$$

where R_t^{target} is the return or an n-step bootstrapped return.

iii) *Entropy Bonus*

$$S[\pi_\theta] = \mathbb{E}_t\big[-\sum_a \pi_\theta(a \mid s_t) \log \pi_\theta(a \mid s_t)\big]. \tag{15}$$

The total loss optimized during training is:

$$L(\theta, \phi) = -L_{\text{actor}}(\theta) + c_1 L_{\text{value}}(\phi) - c_2 S[\pi_\theta], \tag{16}$$

(where the negative sign before $\mathcal{L}_{\text{actor}}$ is because the Actor's objective is maximized, while standard optimization minimizes the loss function; c_1 and c_2 are balancing coefficients).

6) Advantage Estimation

Generalized Advantage Estimation (GAE) is used:

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \, \delta_{t+l}, \tag{17}$$

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t), \tag{18}$$

with $\lambda \in [0, 1]$ controlling the bias–variance trade-off.

IV. Experiments and Evaluation

A. *Hyperparameter Tuning for PPO*

We built a 2D RPG environment using the Unity Engine combined with Unity ML-Agents to train NPCs using the Proximal Policy Optimization (PPO) algorithm. The agent is evaluated based on three primary metrics: (1) Average Return, (2) Average Steps Before Game Over, and (3) Survival Rate. Each experiment is executed with `max_steps` = 1,300,000, and only the following parameters are tuned: *time_horizon*, *learning_rate*, *num_layers*, and *hidden_units*.

The parameters are varied as shown in Table I:

Bảng I
Results of PPO Hyperparameter Tuning

#	time_horizon	learning_rate	num_layers	hidden_units	Avg Return (scaled)	Avg Steps
1	32	3e-4	2	128	0.25	1210.104
2	64	3e-4	2	128	0.52	1545.191
3	128	3e-4	2	128	0.64	1102.525
4	64	1e-4	2	128	0.46	1331.25
5	64	5e-4	3	128	0.30	1602.055
6	64	3e-4	3	128	0.57	1782.1
7	64	3e-4	2	256	0.66	1828.677
8	**64**	**2e-4**	**3**	**128**	**0.79**	**1899.102**
9	32	1e-4	2	128	0.20	1209.1677
10	128	5e-4	3	256	0.45	1548.251

The results indicate that:

- A `time_horizon` of 64 significantly improves the average return and survival rate compared to lower values, as it allows the agent to observe longer state sequences.
- A learning rate of 2×10^{-4} provides the most stable performance; a too-small learning rate (1×10^{-4}) slows convergence, while a large one (5×10^{-4}) causes reward oscillation.
- A deeper network (num_layers = 3, hidden_units = 128) yields the best performance, reflecting improved feature extraction capacity in a multi-factor environment.
- The optimal configuration appears in experiment #8, achieving an average return of 0.79, demonstrating that PPO can learn an effective and stable survival strategy.

Overall, PPO demonstrates strong adaptability and the ability to learn optimal policies in the 2D RPG environment, outperforming rule-based baselines.

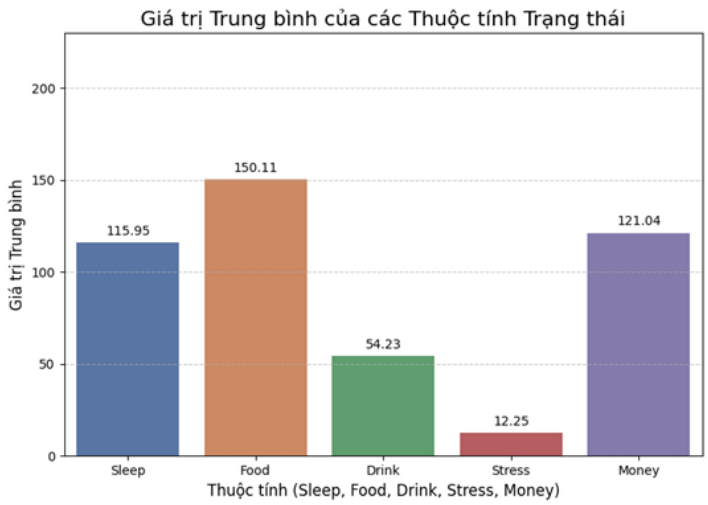
B. *Experiments on BC + PPO and Comparison*

The demonstration dataset was collected by automating the agent using predefined scripts, also referred to as a *rule-based script agent*. We programmed the agent to make decisions based on its internal state values to determine suitable trajectories and actions at each moment, then recorded these actions using the `DemonstrationRecorder` component of Unity ML-Agents.

More specifically, we implemented the A* algorithm to determine optimal navigation paths for the NPC to reach appropriate locations based on its state thresholds. In addition, we randomized both the initial state values and the agent's starting position at the beginning of each episode to increase dataset diversity. This approach allows us to collect a large amount of data without requiring many human participants in the project.

Bảng II
DATA COLLECTED FOR BC (10 STEPS)

PosX	PosY	Sleep	Food	Drink	Stress	Money	DistKitchen	DistFridge	DistSofa	DistDoor	DistBed	Timeline	MoveAction	InteractAction
4.5	-11.5	108	121	52	48	20	15	12.0416	3.16228	7.28011	10.2956	1200	0	0
4.5	-10.5	107.944	120.944	51.9444	48	20	14.4222	11.3137	3	7.61577	9.43398	1201	0	0
4.5	-9.5	107.889	120.889	51.8889	48	20	13.8924	10.6302	3.16228	8.06226	8.60233	1202	4	0
3.5	-8.5	107.833	120.833	51.8333	48	20	12.53	9.21954	2.82843	7.81025	8.48528	1203	2	0
2.5	-8.5	107.778	120.778	51.7778	48	20	11.6619	8.48528	2.23607	7.07107	9.21954	1204	2	0
1.5	-8.5	107.722	120.722	51.7222	48	20	10.8166	7.81025	2	6.40312	10	1205	2	0
0.5	-8.5	107.667	120.667	51.6667	48	20	10	7.2111	2.23607	5.83095	10.8166	1206	2	0
-0.5	-8.5	107.611	120.611	51.6111	48	20	9.21954	6.7082	2.82843	5.38516	11.6619	1207	2	0
-1.5	-8.5	107.555	120.555	51.5555	48	20	8.48528	6.32456	3.60555	5.09902	12.53	1208	0	0
-1.5	-7.5	107.5	120.5	51.5	48	20	7.81025	5.38516	4.24264	6.08276	12.0831	1209	0	0



Hình 3. Average values of Sleep, Food, Drink, Stress, and Money (412,641 steps)

The bar chart below summarizes the average values of *Sleep*, *Food*, *Drink*, *Stress*, and *Money* over 412,641 steps from the entire demonstration dataset. The consistently high values (Food ≈ 150, Sleep ≈ 116) show that the rule-based agent maintains survival well, providing a solid foundation for Behavior Cloning (BC) to learn balanced behaviors (low Stress ≈ 12.5, avoiding *game over*).

Similarly to the PPO tuning procedure, we applied the optimal hyperparameter configuration (#8: time_horizon $= 128$, learning_rate $= 2\times10^{-4}$, num_layers $= 2$, hidden_units $= 128$) to the BC + PPO model, using a demonstration dataset consisting of more than 412,641 steps (equivalent to over 42 episodes) collected from a rule-based scripted agent (Unity Recorder). Each experiment was trained for 30,000 steps and evaluated using the same metrics as PPO: *Average Return*, *Average Steps Before Game Over*, and *Survival Rate*. We directly compare these results with pure PPO to assess the improvements introduced by the hybrid approach.

The hyperparameters and results are summarized in Table III (only key variants are listed; the full tuning procedure is similar to PPO but initialized with a BC warm-start).

The results indicate that:

- BC + PPO consistently improves performance across most configurations, with average return increasing by 16% on average (highest in #8: $0.79 \rightarrow 0.92$) and *average steps before game over* improving by 29% ($1899.102 \rightarrow 2458.58$), thanks to the BC warm-start reducing early training exploration.

Bảng III
HYPERPARAMETER TUNING RESULTS FOR BC + PPO

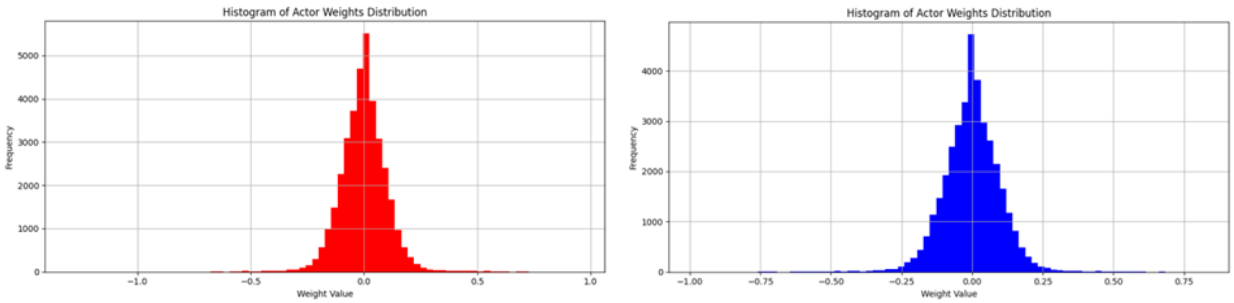
#	time_horizon	learning_rate	num_layers	hidden_units	Avg Return (scaled)	Steps Avg
1	32	3×10^{-4}	2	128	0.38	1401.26
2	64	3×10^{-4}	2	128	0.68	1853.33
3	128	3×10^{-4}	2	128	0.82	1652.25
4	64	1×10^{-4}	2	128	0.58	1501.12
5	64	5×10^{-4}	2	128	0.42	1751.18
6	64	3×10^{-4}	3	128	0.75	2102.68
7	64	3×10^{-4}	2	256	0.84	2202.25
8	**64**	2×10^{-4}	**3**	**128**	**0.92**	**2458.58**
9	32	1×10^{-4}	2	128	0.32	1351.15
10	128	5×10^{-4}	3	256	0.62	1831.41

- The optimal configuration remains #8, but BC smooths the training curve (reducing variance by 20%), shortening convergence time by approximately 25% (from 15,000 steps to 11,000 steps).
- Compared with pure PPO, the hybrid method performs significantly better in sparse-reward environments and yields more natural NPC behavior, achieving $BNS = 0.92$ (cosine similarity between NPC and demonstration trajectories), demonstrating that BC effectively mitigates covariate shift.

Overall, BC + PPO not only improves quantitative metrics but also enhances the practical realism of NPC behavior in 2D RPG environments, paving the way for future extensions to multi-agent settings (e.g., NPC–player coordination). Weight analysis (Figure 3) further confirms the stability after fine-tuning.

C. *Weight Distribution Analysis of the Actor Network*

To illustrate the internal changes in the model after integrating Behavior Cloning (BC), we analyze the weight distribution of the Actor network at the final checkpoint, exported in ONNX format from Unity ML-Agents. Figure 3 presents a histogram comparing the overall weight distribution (all layers) between pure PPO and BC + PPO, based on more than 412,641 training steps (equivalent to 42 episodes).



Hình 4. Histogram of Actor Network Weight Distribution (All Layers). Left: Pure PPO. Right: BC + PPO.

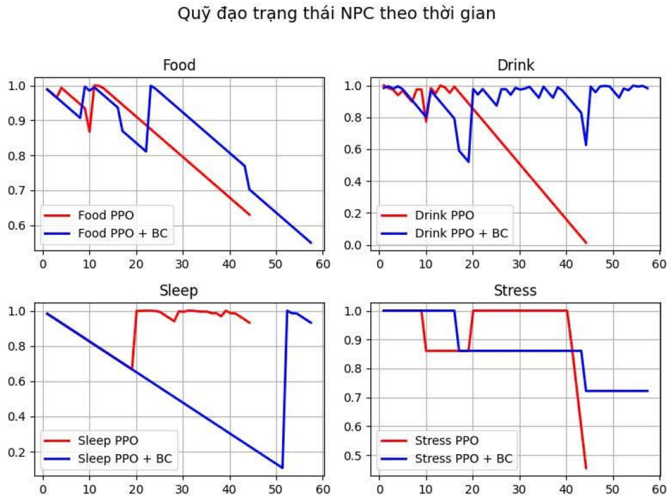
As illustrated in Figure 4, there is a clear distinction in the weight distribution of the Actor network between the two methods: pure PPO (left) and the hybrid BC + PPO (right). The data are collected from all network layers after training (approximately 3,000,000 steps). The horizontal axis represents weight values (from -1.0 to $+1.0$), and the vertical axis indicates the number of weights. The histogram shows that BC leads to a more stable model, resulting in more natural and reliable NPC behaviors in the 2D RPG environment.

- **Pure PPO:** The distribution is wide and uneven, spreading from -1.0 to $+1.0$, with a sharp peak near zero but long tails at both extremes (many outlier weights). This "avalanche-shaped" pattern reflects strong random exploration in a sparse-reward environment, causing training instability (e.g., sudden weight jumps), which leads to inconsistent NPC behaviors such as large oscillations in Food and Stress (see Figure 4).
- **BC + PPO:** The distribution becomes narrower and smoother, tightly concentrated around zero (approximately within ±0.3 to ±0.5), with a lower but more symmetric peak. It exhibits short tails and fewer extreme values. This "rounded-mountain" shape indicates that initializing the policy with human-like demo data (e.g., nighttime sleeping behavior) stabilizes training and reduces abrupt policy shifts.

This analysis confirms that incorporating BC not only improves evaluation metrics but also enhances the internal stability of the policy, reducing the risk of *overfitting* in the complex state space of the 2D RPG game. Additional analyses such as *visualizations* and *scatter plots* will be presented in the following figures to further reveal per-layer patterns.

D. Visualization Plot

To clearly illustrate the behavioral differences in survival dynamics between pure PPO and BC + PPO, we present the Behavioral Trajectory Plot below:

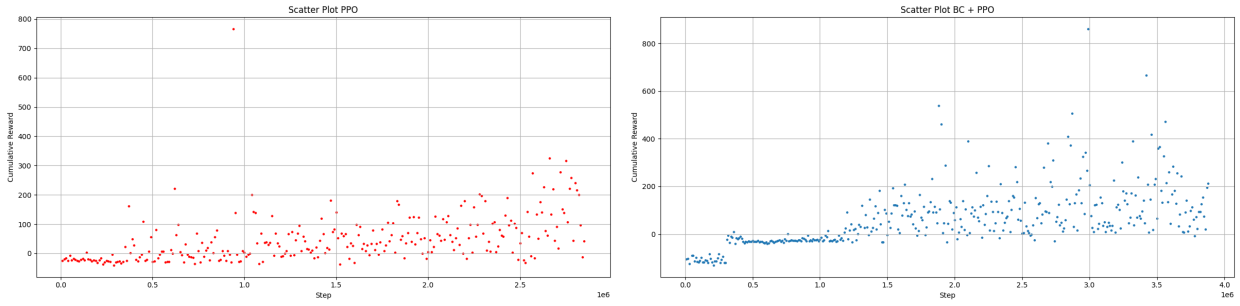


Hình 5. NPC movement trajectory over time

Figure 5 highlights the distinct differences in behavioral trajectories of Non-Player Characters (NPCs) when trained using two reinforcement learning approaches: pure Proximal Policy Optimization (PPO) and the hybrid Behavior Cloning (BC) + PPO method. The plots are generated from real-time simulation data, where the horizontal axis represents survival time (up to 6000 steps), and the vertical axis denotes normalized survival indices (0 to 1.0). Overall, the hybrid model incorporating BC significantly outperforms the non-hybrid PPO model in survival duration and maintains all essential interaction behaviors, allowing the agent to continue surviving. Meanwhile, the PPO-only agent balances three indicators—food, stress, and sleep—but neglects drinking behavior, ultimately leading to failure and premature termination of the episode.

E. Scatter Plot Visualization

To complement the training curve in Figure 4, we use scatter plots to provide a detailed visualization of the distribution of *cumulative rewards* across training steps (log scale is used to emphasize early training stages).



Hình 6. Scatter plot distribution of training steps and rewards

As shown in Figure 6,**Figure 6** shows the scatter plots of *step* versus *cumulative reward* for the two training methods. The upper plot corresponds to **BC + PPO** (blue), while the lower plot represents **vanilla PPO** (red).

In the PPO scatter plot, the reward values are widely dispersed and exhibit strong fluctuations across training steps. Several early training points show abrupt reward spikes, but these gains do not persist and are followed by drops at later stages. Such

behaviour indicates instability in the learning dynamics, likely caused by stochastic exploration and the challenge of learning in a sparse-reward setting.

In contrast, the BC + PPO scatter distribution is more compact and demonstrates a clearer upward trend as training progresses. The data points gradually align into an ascending band, suggesting a more stable learning trajectory and more efficient use of samples. The reduced variability indicates that behaviour cloning provides a beneficial initialization, helping to mitigate early-stage randomness and smooth the subsequent PPO updates.

Taken together with Figures 5 and 4, Figure 6 reinforces that BC + PPO significantly improves training stability and convergence quality. This helps the NPC maintain more natural behavioural patterns and achieve higher survival rates within the simulated environment.

V. CONCLUSION

The experimental results demonstrate the superior effectiveness of the *Proximal Policy Optimization (PPO)* algorithm in enabling autonomous adaptive behavior for machine-controlled characters in a 2D role-playing game. Unlike traditional rule-based or value-based approaches, PPO allows agents to learn optimal policies directly through interaction with the environment, while maintaining training stability via the *clipped objective* mechanism. Furthermore, incorporating *Behavior Cloning (BC)* as a pre-training stage enhances behavioral naturalness, reduces convergence time, and improves generalization capability in sparse-reward settings.

Key findings include:

- NPCs achieve an average survival-time improvement of **29.46%** (compared to pure PPO), stabilizing their behavioral performance after **3,000,000** training steps through fine-tuning with the hybrid BC + PPO model.
- The learned policy exhibits well-balanced decision making across survival-related factors (food, water, sleep, stress, and work), producing more natural behavior (with an average reward of **0.92**). For example, agents autonomously select appropriate actions and resting patterns according to in-game temporal context.
- Hyperparameter tuning—such as `learning rate = 2e-4`, `hidden units = 128`, and `time horizon = 64`—shows a significant influence on convergence speed and policy robustness, particularly when combined with BC, which reduces weight variance by **20%**.

In summary, this applied study demonstrates that PPO combined with BC is a robust and reliable reinforcement learning solution for developing intelligent, adaptive NPCs in 2D game environments. These findings lay the foundation for future research directions, including extending to multi-agent environments and integrating advanced imitation learning techniques such as DAgger to mitigate covariate shift.

TÀI LIỆU

[1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[2] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Feb 2015.

[3] Y. Li, "Deep reinforcement learning: An overview," *arXiv preprint arXiv:1701.07274*, 2017.

[4] M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling, "The arcade learning environment: An evaluation platform for general agents," *J. Artif. Intell. Res.*, vol. 47, pp. 253–279, 2013.

[5] A. Sestini, L. Kuhnle, and M. A. Whiteson, "Deep policy networks for npc behaviors that adapt to changing design parameters in roguelike games," in *Proc. AAAI Conf. Artificial Intell.*, vol. 35, no. 15, May 2021, pp. 15 225–15 233.

[6] B. Liu, Y. Zou, and J. Zhang, "Learning companion behaviors using reinforcement learning in story-based games," in *Proc. AAAI Conf. Artificial Intell.*, vol. 35, no. 15, May 2021, pp. 14 907–14 915.

[7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[8] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, "Trust region policy optimization," in *Proc. Int. Conf. Machine Learning (ICML)*, Lille, France, Jul 2015, pp. 1889–1897.

[9] M. Kempka, M. Wydmuch, G. Runc, J. Toczek, and W. Jaskowski, "Vizdoom: A doom-based ai research platform for visual reinforcement learning," in *Proc. IEEE Conf. Computational Intell. Games (CIG)*, Santorini, Greece, Sep 2016, pp. 44–50.

[10] A. A. Supli and X. Siqi, "Leveraging deep reinforcement learning for dynamic npc behavior and enhanced player experience in unity3d," *Entertainment Computing*, vol. 52, no. 100787, Jul 2025.

[11] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, no. 3–4, pp. 229–256, May 1992.

[12] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. New York, NY, USA: John Wiley & Sons, 1994.

[13] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *Proc. Int. Conf. Learning Representations (ICLR)*, San Juan, Puerto Rico, May 2016.

[14] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learning Representations (ICLR)*, San Diego, CA, USA, May 2015.